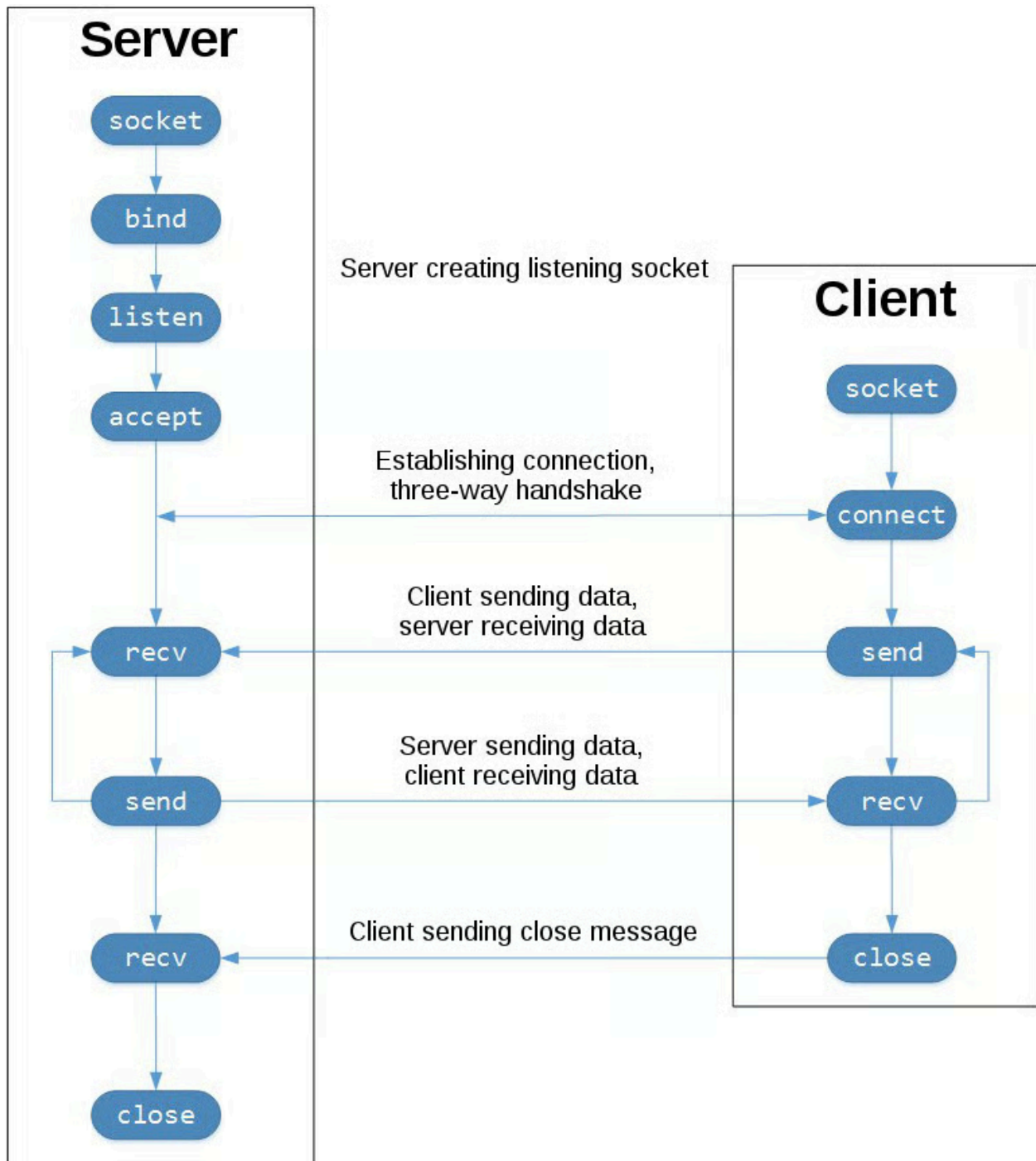


Socket Programming -



1. The `bind()` function tells the server which IP address and port number it will use to listen for incoming connections.

Good question 👍

Let's answer this **slowly and clearly**, like a class note.

I'll give you **the socket functions**, what they do, and **who uses them** — **no confusion**, no extra stuff.

Socket Functions in Python (Simple List)

◆ 1. `socket()`

What it does:

👉 Creates a socket (communication tool)

Used by: Server & Client

```
s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
```

◆ 2. `bind()`

What it does:

👉 Gives the server its IP address and port

Used by: Server only

```
s.bind(("127.0.0.1", 9999))
```

◆ 3. `listen()`

What it does:

👉 Tells server to wait for connections

Used by: Server only

```
s.listen(5)
```

◆ 4. `accept()`

What it does:

👉 Accepts a client connection

Used by: Server only

```
conn, addr = s.accept()
```

◆ 5. `connect()`

What it does:

👉 Client connects to server

Used by: Client only

```
s.connect(("127.0.0.1", 9999))
```

◆ 6. `connect_ex()`

What it does:

👉 Tries to connect and returns result

(does NOT crash on failure)

Used by: Client (port scanner)

```
result = s.connect_ex(("127.0.0.1", 80))
```

◆ 7. `send()`

What it does:

👉 Sends data

Used by: Server & Client

```
s.send(b"Hello")
```

◆ 8

Function	Purpose
<code>gethostname()</code>	Get your computer name
<code>gethostbyname()</code>	Name → IP
<code>gethostbyaddr()</code>	IP → Name
<code>getfqdn()</code>	Full domain name
<code>getservbyname()</code>	Service → Port
<code>getservbyport()</code>	Port → Service
<code>inet_aton()</code>	IP → binary
<code>inet_ntoa()</code>	binary → IP
<code>setdefaulttimeout()</code>	Global timeout

Port Scanner -

1. What this program does (overall summary)

This program is a **TCP connect port scanner** written using Python's `socket` module.

Its workflow is:

1. Take a **host** (IP address or domain name) from the user
2. Take a **range of ports** to scan
3. Convert the domain name into an IP address if needed
4. For each port in the range:

- Attempt a TCP connection
- Decide whether the port is OPEN or CLOSED
- Identify the common service running on that port

5. Display the results in a clean table format

This is the same scanning technique used by tools like **nmap -sT** (TCP connect scan).

2. Input handling and host resolution

```
host =input("Enter the ip address or domain name you want to scan:")
```

The user can enter:

- An IP address (example: `127.0.0.1`)
- A domain name (example: `scanme.nmap.org`)

```
ip = socket.gethostbyname(host)
```

Function explanation:

- `socket.gethostbyname()` resolves a **domain name to an IPv4 address**
- If an IP address is already given, it simply returns it unchanged

This allows the scanner to work for **both domains and IPs**.

3. Exception: `socket.gaierror`

```
except socket.gaierror:  
    ip = host
```

Why this exception exists:

- `socket.gaierror` occurs when DNS resolution fails

- Example causes:
 - Invalid domain name
 - DNS server not reachable

Why you handle it:

- If DNS fails, the program assumes the input is already an IP address
- This prevents the program from crashing

This makes the scanner more robust.

4. Port range input

```
start_port = int(input("Enter the starting port:"))
stop_port = int(input("Enter the last port:"))
```

Key point:

- `input()` returns a string
- `int()` converts it into an integer so it can be used in `range()`

5. Looping through ports

```
for i in range(start_port, stop_port + 1):
```

Purpose:

- Iterates through every port in the user-defined range
- `+1` ensures the last port is included

6. Socket creation and configuration

```
scan_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
```

Function explanation:

- `AF_INET` → IPv4 addressing
- `SOCK_STREAM` → TCP protocol

A **new socket is created for each port**.

This is important because sockets cannot be reused reliably after a connection attempt.

```
scan_socket.settimeout(1)
```

Function explanation:

- Limits how long the program waits for a response
- Prevents the scan from hanging on filtered or slow ports

7. Connecting to the port

```
result = scan_socket.connect_ex((ip, i))
```

Function explanation:

- Attempts a TCP connection to `(ip, port)`
- Does not raise an exception on failure
- Returns a status code:
 - `0` → connection successful (port OPEN)
 - non-zero → connection failed (port CLOSED or FILTERED)

This function is ideal for port scanning because it allows graceful error handling.

8. Exception: `socket.gaierror` during connection

```
except socket.gaierror: print("Enter correct domain name") break
```

Why this exception exists:

- Occurs if the resolved IP or hostname is invalid
- Indicates a serious input error

Why `break` is used:

- There is no point continuing the scan if the host itself is invalid
-

9. Service identification

```
service = socket.getservbyport(i)
```

Function explanation:

- Maps well-known ports to standard service names
 - Examples:
 - 22 → ssh
 - 80 → http
 - 443 → https
-

10. Exception: `OSError` for unknown services

```
except OSError:  
    service = "unknown"
```

Why this exception exists:

- Many ports do not have a registered service
- `getservbyport()` raises `OSError` in that case

Why you handle it:

- Prevents the program from crashing
 - Allows the scan to continue normally
-

11. Port status determination

```
if result ==0:  
    status ="OPEN"else:  
    status ="CLOSED"
```

Logic explanation:

- Successful TCP handshake means a service is listening
- Failed handshake means nothing is listening or access is blocked

12. Output formatting

```
print(f"{i:<5} |{status:<6} |{service}")
```

Explanation:

- Uses formatted strings to align columns
- `<5` and `<6` ensure clean, readable output
- Produces a table-like display similar to professional tools

13. Closing the socket

```
scan_socket.close()
```

Why this is important:

- Releases system resources
- Prevents file descriptor exhaustion
- Ensures stability during large scans

Nmap Scanner -

Nice progression — this is the **next logical step after writing your own scanner**.

I'll **summarize what this code does**, then **explain every important function and concept used**, and finally point out **small issues/improvements** so you fully understand it.

1 What this program does (high-level summary)

This script uses **python-nmap**, which is a **Python wrapper around the Nmap tool**.

Your program:

1. Takes an IP address from the user
2. Asks what type of scan to run
3. Runs Nmap with different scan flags based on the choice
4. Displays:
 - Nmap version
 - Scan information
 - Host status (up/down)
 - Protocols detected
 - Open ports

So instead of manually using sockets, you are now **automating Nmap using Python**.

2 Line-by-line conceptual explanation

Importing the library

```
import nmap
```

- Imports the **python-nmap** module
 - This module internally calls the **Nmap binary**
 - Nmap **must be installed on the system**
-

Creating a PortScanner object

```
scanner = nmap.PortScanner()
```

This object:

- Acts as a **controller for Nmap**
- Runs scans
- Stores scan results in a dictionary-like structure

Think of it as:

| "Python interface to Nmap"

Taking IP address input

```
ip_addr = int(input("Please enter the Ip address you want to scan: "))
```

Conceptually:

- Takes user input

Problem:

- IP addresses are **strings**, not integers
- This should be:

```
ip_addr = input("Please enter the IP address you want to scan: ")
```

Otherwise, the code will fail for real IPs like `192.168.1.1`.

Displaying entered IP

```
print("The IP you entered us: ", ip_addr)
```

- Simple confirmation output

Asking for scan type

```
resp = input("""
Please enter the type of scan you want to run
1) SYN ACK Scan
2) UDP Scan
3) Comprehensive Scan
""")
```

- Multiline input prompt
- Stores the user's choice as a string ('1', '2', '3')

Showing selected option

```
print('You have selected option: ', resp)
```

3 Scan types and what they really mean

OPTION 1: SYN Scan

```
scanner.scan(ip_addr, '1-1024', '-v -sS')
```

Function used

```
scanner.scan(hosts, ports, arguments)
```

- `hosts` → target IP
- `ports` → range of ports
- `arguments` → Nmap flags

Flags explained

- `sS` → SYN scan (half-open scan)
- `v` → verbose output

This scan:

- Sends SYN packets
- Does not complete TCP handshake
- Fast and stealthy
- Requires **root privileges**

OPTION 2: UDP Scan

```
scanner.scan(ip_addr, '1-1024', '-v -sU')
```

Flags

- `sU` → UDP scan
- Slower than TCP
- Used to find services like DNS, SNMP

Note:

- UDP results are often `open|filtered`

OPTION 3: Comprehensive Scan

```
scanner.scan(ip_addr, '1-1024', '-v -A')
```

A means

Enables:

- OS detection
- Version detection
- Script scanning
- Traceroute

This is a **very aggressive and noisy scan**.

4 Understanding the output functions

Nmap version

```
scanner.nmap_version()
```

Returns:

- Nmap version
- Useful for debugging and logging

Scan information

```
scanner.scaninfo()
```

Returns:

- Scan type
- Protocols used
- Services scanned

Example:

```
{'tcp': {'method': 'syn', 'services': '1-1024'}}
```

Host status

```
scanner[ip_addr].state()
```

Returns:

- `'up'`
- `'down'`

This tells whether the host is reachable.

Protocols detected

```
scanner[ip_addr].all_protocols()
```

Returns:

```
['tcp']  
['tcp', 'udp']
```

Open ports

```
scanner[ip_addr]['tcp'].keys()
```

- Returns open TCP ports
- Example:

```
dict_keys([22, 80, 443])
```

Note:

In UDP scan, this should be:

```
scanner[ip_addr]['udp'].keys()
```

Your code still uses `'tcp'` for UDP scan, which is a logic bug.

5 What this program demonstrates conceptually

You have now covered **three levels of scanning**:

1. Manual socket-based scanner

- Low-level
- Full control
- Educational

2. Nmap automation using Python

- High-level
- Professional
- Real-world tooling

3. Scan strategy selection

- TCP SYN
 - UDP
 - Aggressive enumeration
-

6 Important issues to fix (learning-focused)

Issue 1: IP should not be int

Fix:

```
ip_addr = input("Please enter the IP address you want to scan: ")
```

Issue 2: UDP scan output key

Fix:


```
scanner[ip_addr]['udp'].keys()
```

Issue 3: Root privilege requirement

- `sS`, `sU`, `A` require **sudo**
- Without sudo, Nmap will silently downgrade scans

7 Final takeaway

- Your **socket scanner taught you how scanning works**
- Your **Nmap scanner teaches you how professionals automate scans**
- You now understand:
 - TCP vs UDP scanning
 - SYN scans vs full connect scans
 - Why Nmap is powerful

Port Scanner using Banner

Below is a **clear summary** of your banner-grabbing program, followed by a **detailed explanation of each function used** and **why the exceptions are handled**. This mirrors the structure you asked for earlier.

1. What this program does (overall summary)

This program performs **banner grabbing**, which is a technique used to collect **identification information from a network service** running on a specific IP address and port.

The program:

1. Takes a **target IP address** and **port number** from the user
2. Creates a TCP connection to that service
3. Receives data sent by the service (the banner)
4. Displays the banner information if available
5. Handles errors and timeouts safely
6. Closes the socket cleanly

Banner grabbing is commonly used to identify:

- Service type (HTTP, FTP, SSH, SMTP)
 - Software name
 - Version information
-

2. Program structure

The program is divided into two logical parts:

- `bannerGrabber()` → performs the actual network operation
- `main()` → handles user input and calls the function

The `if __name__ == "__main__":` block ensures correct execution.

3. `bannerGrabber()` function

```
def bannerGrabber(inputIp, inputPort):
```

Purpose:

- Connects to a remote service
 - Attempts to receive and display banner data
-

Socket creation

```
s = socket.socket()
```

Explanation:

- Creates a TCP socket
- Defaults to IPv4 (`AF_INET`) and TCP (`SOCK_STREAM`)
- This socket will be used to communicate with the target service

Establishing a connection

```
s.connect((inputIp, inputPort))
```

Explanation:

- Initiates a TCP connection to the given IP and port
- If the service is not listening or blocked, an exception is raised

Setting timeout

```
s.settimeout(5)
```

Explanation:

- Limits blocking operations (like `recv()`) to 5 seconds
- Prevents the program from hanging indefinitely
- Essential for network reconnaissance tools

Receiving banner data

```
bannerData = s.recv(1024).decode()
```

Explanation:

- `recv(1024)` reads up to 1024 bytes from the service

- Many services send a banner immediately after connection
 - `.decode()` converts bytes to a readable string
-

Checking received data

```
if bannerData:
    print(f"Banner details: {bannerData}")
else:
    print("Could not connect to the service.")
```

Explanation:

- If data is received, it is printed
 - If no data is sent, the service may require a request first
-

4. Exception handling and why it is needed

`socket.timeout`

```
except socket.timeout:
    print("Request timeout")
```

Reason:

- Triggered when the service does not respond within the timeout period
 - Common for firewalled or filtered services
 - Prevents the program from freezing
-

`socket.error`

```
except socket.error as e:
    print(f"Socket error: {e}")
```

Reason:

- Catches general socket-related errors
- Examples:
 - Connection refused
 - Network unreachable
 - Invalid port

Printing the error helps in debugging.

finally block

```
finally:  
    s.close()
```

Reason:

- Ensures the socket is always closed
 - Prevents resource leaks
 - Executes regardless of success or failure
-

5. **main()** function

```
def main():
```

Purpose:

- Handles user interaction
 - Keeps the logic separate from networking code
-

User input

```
inputIp = str(input("Enter the target ip address: "))  
inputPort = int(input("Enter the port: "))
```

Explanation:

- IP address is taken as a string
 - Port is converted to integer for socket operations
-

Function call

```
bannerGrabber(inputIp, inputPort)
```

Explanation:

- Passes user input to the banner grabbing function
 - Keeps code modular and reusable
-

6. Program entry point

```
if __name__ == "__main__":  
    main()
```

Explanation:

- Ensures `main()` runs only when the script is executed directly
 - Allows the file to be imported as a module without running immediately
-